

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
Error Handling



Disusun Oleh:
Bagas Efriliyanto Setiawan (105224049)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pengembangan sistem informasi layanan publik, seperti sistem reservasi tiket kereta api, menuntut tingkat stabilitas aplikasi yang sangat tinggi. Masalah operasional yang sering ditemui di lapangan umumnya bersumber dari kesalahan pengguna (human error) saat memasukkan data pada antarmuka CLI (Command Line Interface), seperti kesalahan tipe data input atau format data personal yang tidak valid. Jika tidak ditangani dengan baik, kesalahan-kesalahan tersebut dapat menyebabkan aplikasi mengalami crash atau berhenti mendadak. Dalam proyek perancangan sistem "JAVA EXPRESS" ini, fokus utama terletak pada implementasi mekanisme Error Handling dan Custom Exception yang ketat. Melalui pemisahan antara Unchecked Exception untuk validasi data personal (NIK) dan Checked Exception untuk logika bisnis ketersediaan rute serta kapasitas kursi, sistem dipastikan mampu mengisolasi error. Dengan perlindungan blok try-catch-finally dan pembersihan input buffer secara berkala, ekosistem reservasi ini dirancang untuk menjadi kebal terhadap skenario infinite loop, interaktif, serta tetap responsif dalam memberikan pesan error yang informatif bagi pengguna..

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kodeKereta	String	Menyimpan identitas unik kode kereta api (e.g., "K01").
2	namaKereta	String	Menyimpan nama komersial kereta api (e.g., "Argo Bromo").
3	rute	String	Menyimpan informasi rute asal dan tujuan perjalanan kereta.
4	sisaKursi	int	Menyimpan jumlah kuota sisa kursi kosong yang tersedia pada kereta.
5	daftarKereta	ArrayList<Kereta>	Menyimpan kumpulan seluruh objek kereta api di dalam memori sistem.
6	pilihanMenu	int	Menyimpan angka pilihan menu utama yang dimasukkan oleh pengguna.
7	input	Scanner	Objek utilitas untuk membaca data masukan dari papan ketik (<i>keyboard</i>).
8	kode / nik / nama	String	Variabel temporer untuk menampung input data formulir penumpang saat memesan tiket.
9	jumlah	int	Variabel temporer untuk menampung jumlah kuota tiket yang ingin dipesan.

III. Dokumentasi dan Pembahasan Code

1. DataPenumpangTidakValidException.java

```
1 // Unchecked Exception untuk validasi data personal (NIK)
2 public class DataPenumpangTidakValidException extends RuntimeException {
3     public DataPenumpangTidakValidException(String message) {
4         super(message);
5     }
6 }
```

Kelas ini merupakan *custom exception* yang didesain khusus untuk menangani kesalahan pada manipulasi data personal penumpang, seperti format NIK yang tidak sesuai standar.

Kelas ini dibuat dengan mengekstensi kelas `RuntimeException`. Hal ini menjadikannya bertipe *Unchecked Exception*, yang berarti compiler tidak memaksa metode pemanggil untuk menangkapnya secara eksplisit menggunakan blok `try-catch` atau mendeklarasikannya di klausul `throws`. Sifat ini sangat cocok untuk memvalidasi kesalahan logika atau input data krusial yang bersifat mutlak.

Di dalam konstruktor, kita menggunakan kata kunci `super(message)` untuk meneruskan string pesan kesalahan ke kelas induk (`RuntimeException`), sehingga pesan tersebut dapat dicetak dengan rapi saat terjadi error di terminal.

2. RuteTidakDitemukanException.java

```
1 // Checked Exception untuk validasi kode/rute kereta
2 public class RuteTidakDitemukanException extends Exception {
3     public RuteTidakDitemukanException(String message) {
4         super(message);
5     }
6 }
```

kelas ini mempresentasikan kegagalan pada pemenuhan logika bisnis sistem, yaitu ketika data perjalanan kereta yang dicari tidak eksis di memori. Kelas ini mengekstensi kelas dasar `Exception`. Konsekuensinya, kelas ini berstatus sebagai *Checked Exception*, di mana compiler Java akan memaksa pengembang untuk menangani potensi error ini di blok penanganan atau secara eksplisit mendeklarasikannya pada *method signature* menggunakan keyword `throws`. Hal ini menjamin bahwa setiap

kali pencarian rute dilakukan, sistem selalu dipaksa bersiap menghadapi skenario terburuk.

3. TiketHabisException.java



```
1 // Checked Exception untuk validasi sisa kursi dengan informasi tambahan
2 public class TiketHabisException extends Exception {
3     private String namaKereta;
4     private int sisaKursi;
5
6     public TiketHabisException(String namaKereta, int sisaKursi) {
7         super("Pemesanan gagal! Tiket untuk kereta " + namaKereta + " tidak mencukupi.");
8         this.namaKereta = namaKereta;
9         this.sisaKursi = sisaKursi;
10    }
11
12    public String getNamaKereta() {
13        return namaKereta;
14    }
15
16    public int getSisaKursi() {
17        return sisaKursi;
18    }
19 }
```

Exception ini dirancang untuk menangani kondisi *overcapacity*, yaitu saat jumlah pesanan tiket melebihi sisa bangku kosong yang tersedia pada jadwal terkait. Sesuai instruksi soal, exception ini wajib membawa informasi aktual mengenai kondisi objek kereta saat transaksi gagal. Oleh karena itu, di dalam kelas ini ditambahkan dua atribut privat baru: String namaKereta dan int sisaKursi. Nilai dari nama kereta dan sisa kursi diisi secara otomatis melalui parameter konstruktor saat exception ini dilempar (throw). Untuk mengambil data tersebut di luar kelas tanpa merusak keamanan data, disediakan dua metode selektor/getter: getNamaKereta() dan getSisaKursi().

4. Kereta.java

```

1 // Model data untuk objek Kereta Api
2 public class Kereta {
3     private String kodeKereta;
4     private String namaKereta;
5     private String rute;
6     private int sisaKursi;
7
8     public Kereta(String kodeKereta, String namaKereta, String rute, int sisaKursi) {
9         this.kodeKereta = kodeKereta;
10        this.namaKereta = namaKereta;
11        this.rute = rute;
12        this.sisaKursi = sisaKursi;
13    }
14
15    public String getKodeKereta() { return kodeKereta; }
16    public String getNamaKereta() { return namaKereta; }
17    public String getRute() { return rute; }
18    public int getSisaKursi() { return sisaKursi; }
19
20    public void kurangiKursi(int jumlah) {
21        this.sisaKursi -= jumlah;
22    }
23 }

```

Kelas ini bertindak sebagai cetakan blueprint (model data) untuk merepresentasikan entitas fisik dari kereta api yang beroperasi di dalam sistem "JAVA EXPRESS". Seluruh atribut seperti kodeKereta, namaKereta, rute, dan sisaKursi diberi hak akses private. Ini dilakukan untuk mencegah kelas luar memodifikasi data internal objek secara ilegal. Akses data dijembatani secara aman menggunakan fungsi Getter. Di dalam kelas ini disediakan metode kurangiKursi(int jumlah). Metode ini tidak menggunakan setter standar, melainkan sebuah fungsi logika bisnis terarah untuk mengurangi kuota kursi aktual setelah proses pemesanan dinyatakan valid.

5. SistemReservasi.java

```

1  import java.util.ArrayList;
2
3  // Pusat kontrol reservasi dan logika bisnis sistem
4  public class SistemReservasi {
5      private ArrayList<Kereta> daftarKereta;
6
7      public SistemReservasi() {
8          daftarKereta = new ArrayList<>();
9          initData();
10     }
11
12     // Mengisi data awal ke memori secara otomatis
13     private void initData() {
14         daftarKereta.add(new Kereta("K01", "Argo Bromo", "JKT - SBY", 50));
15         daftarKereta.add(new Kereta("K02", "Parahyangan", "JKT - BDG", 15));
16     }
17
18     public void tampilkanJadwal() {
19         System.out.println("\n=== JADWAL KERETA JAVA EXPRESS ===");
20         System.out.printf("%-12s %-15s %-15s %-10s\n", "KODE KERETA", "NAMA KERETA", "RUTE", "SISA KURSI");
21         System.out.println("-----");
22         for (Kereta k : daftarKereta) {
23             System.out.printf("%-12s %-15s %-15s %-10d\n",
24                 k.getKodeKereta(), k.getNamaKereta(), k.getRute(), k.getSisaKursi());
25         }
26     }
27
28     // Metode pemesanan tiket dengan deklarasi 'throws' secara eksplisit
29     public void pesanTiket(String kode, String nik, String nama, int jumlahTiket)
30         throws RuteTidakDitemukanException, TiketHabisException {
31
32         // Validasi NIK (Must be 16 digits of numbers)
33         if (nik.length() != 16) {
34             throw new DataPenumpangTidakValidException("Format NIK Salah: Harus tepat 16 karakter!");
35         }
36         if (!nik.matches("[0-9]+")) {
37             throw new DataPenumpangTidakValidException("Format NIK Salah: NIK hanya boleh berisi angka!");
38         }
39
40         // Cari kereta berdasarkan kode
41         Kereta keretaDipilih = null;
42         for (Kereta k : daftarKereta) {
43             if (k.getKodeKereta().equalsIgnoreCase(kode)) {
44                 keretaDipilih = k;
45                 break;
46             }
47         }
48
49         if (keretaDipilih == null) {
50             throw new RuteTidakDitemukanException("Error: Kereta dengan kode \"" + kode + "\" tidak ditemukan!");
51         }
52
53         // Validasi ketersediaan kursi
54         if (jumlahTiket > keretaDipilih.getSisaKursi()) {
55             throw new TiketHabisException(keretaDipilih.getNamaKereta(), keretaDipilih.getSisaKursi());
56         }
57
58         // Proses modifikasi data jika semua validasi lolos
59         keretaDipilih.kurangiKursi(jumlahTiket);
60         System.out.println("\n=====");
61         System.out.println("      RESERVASI TIKET BERHASIL DICATAT!      ");
62         System.out.println("=====");
63         System.out.println("Nama Penumpang : " + nama);
64         System.out.println("Kereta Api      : " + keretaDipilih.getNamaKereta());
65         System.out.println("Rute Perjalanan: " + keretaDipilih.getRute());
66         System.out.println("Jumlah Tiket   : " + jumlahTiket + " kursi");
67         System.out.println("Sisa Kursi Kini: " + keretaDipilih.getSisaKursi());
68         System.out.println("=====");
69     }
70 }

```

Memanfaatkan koleksi objek `ArrayList<Kereta>`, sistem ini secara otomatis memesan slot memori dan melakukan instansiasi dua data awal kereta api ("Argo Bromo" dan "Parahyangan") saat konstruktor kelas ini dipicu pertama kali. Metode ini menerima parameter komplit dan mendeklarasikan potensi kesalahannya secara eksplisit menggunakan `throws RuteTidakDitemukanException, TiketHabisException`. Dibagi 3 layer filter :

- 1) Filter 1 (NIK): Mengecek panjang karakter string NIK dan memanfaatkan fungsi `regex matches` untuk mendeteksi keberadaan karakter alfabet. Jika

melanggar aturan, instansiasi `DataPenumpangTidakValidException` langsung dilempar.

- 2) Filter 2 (Kode Kereta): Melakukan iterasi pencarian objek di dalam `ArrayList`. Jika pencarian selesai dan objek tetap bernilai null, maka `RuteTidakDitemukanException` diledakkan.
- 3) Filter 3 (Kuota Kursi): Membandingkan variabel parameter `jumlahTiket` dengan properti internal `sisaKursi` milik objek kereta yang terpilih. Jika kuota tidak mencukupi, program melempar `TiketHabisException` seraya mengirimkan data objek kereta tersebut.

6. Main.Java


```

1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  // Kelas Utama untuk Interface CLI dan Penanganan Exception Utama
5  public class Main {
6      public static void main(String[] args) {
7          Scanner input = new Scanner(System.in);
8          SistemReservasi sistem = new SistemReservasi();
9          int pilihanMenu = 0;
10
11          System.out.println("Selamat Datang di Sistem Reservasi \"JAVA EXPRESS\"");
12
13          do {
14              System.out.println("\n===== MENU UTAMA =====");
15              System.out.println("1. Lihat Jadwal & Sisa Kursi");
16              System.out.println("2. Pesan Tiket Kereta");
17              System.out.println("3. Keluar Aplikasi");
18              System.out.print("Pilih menu (1-3): ");
19
20              try {
21                  pilihanMenu = input.nextInt();
22                  input.nextLine(); // Bersihkan sisa newline di buffer
23
24                  switch (pilihanMenu) {
25                      case 1:
26                          sistem.tampilkanJadwal();
27                          break;
28
29                      case 2:
30                          System.out.println("\n--- FORMULIR PEMESANAN TIKET ---");
31                          System.out.print("Masukkan Kode Kereta (e.g. K01): ");
32                          String kode = input.nextLine().trim();
33
34                          System.out.print("Masukkan NIK Penumpang      : ");
35                          String nik = input.nextLine().trim();
36
37                          System.out.print("Masukkan Nama Penumpang    : ");
38                          String nama = input.nextLine().trim();
39
40                          System.out.print("Masukkan Jumlah Tiket      : ");
41                          int jumlah = input.nextInt();
42                          input.nextLine(); // Bersihkan buffer input setelah membaca angka
43
44                          try {
45                              sistem.pesanTiket(kode, nik, nama, jumlah);
46                          } catch (DataPenumpangTidakValidException e) {
47                              System.out.println("\n[KOSMETIK ERROR - UNCHECKED]");
48                              System.out.println("Gagal Gara-gara Data: " + e.getMessage());
49                          } catch (RuteTidakDitemukanException e) {
50                              System.out.println("\n[KOSMETIK ERROR - CHECKED]");
51                              System.out.println("Gagal Gara-gara Rute: " + e.getMessage());
52                          } catch (TiketHabisException e) {
53                              System.out.println("\n[KOSMETIK ERROR - CHECKED]");
54                              System.out.println(e.getMessage());
55                              System.out.println("Info Aktual -> Kereta: " + e.getNamaKereta()
56                                  + " | Sisa Kursi Tersedia: " + e.getSisaKursi() + " kursi.");
57                          }
58                          break;
59
60                      case 3:
61                          System.out.println("Memproses penutupan sesi sistem...");
62                          break;
63
64                      default:
65                          System.out.println("Peringatan: Pilihan menu tidak valid! Silakan masukkan angka 1-3.");
66                  }
67
68              } catch (InputMismatchException e) {
69                  System.out.println("\n[ERROR INPUT CRITICAL]");
70                  System.out.println("Waduh! Input harus berupa angka ya, jangan memasukkan huruf atau simbol.");
71                  input.nextLine(); // Penting: Membersihkan buffer input dari token salah agar terhindar dari infinite loop
72              } finally {
73                  // Blok penanganan error/log penutupan yang dijamin pasti tereksekusi saat keluar
74                  if (pilihanMenu == 3) {
75                      System.out.println("-----");
76                      System.out.println("Blok Finally: Terhubung ke database... Sesi ditutup dengan aman.");
77                      System.out.println("Terima kasih telah menggunakan layanan JAVA EXPRESS! Sampai jumpa.");
78                      System.out.println("-----");
79                  }
80              }
81
82          } while (pilihanMenu != 3);
83
84          input.close();
85      }
86  }

```

Kelas ini bertindak sebagai eksekutor utama, penyedia antarmuka CLI (Command Line Interface), sekaligus tameng pelindung (*Error Handling*) terluar yang berinteraksi langsung dengan pengguna. Kesalahan umum pada penggunaan objek Scanner di dalam perulangan do-while adalah macetnya sistem saat menerima jenis data yang salah (misal mengetik huruf pada metode `.nextInt()`). Di dalam blok `catch InputMismatchException`, kita sengaja menaruh perintah `input.nextLine()`. Perintah ini berfungsi vital sebagai penyapu atau pembersih token data rusak yang terjebak di dalam *input buffer* memori, sehingga program langsung pulih dan terhindar dari *crash* ataupun *infinite loop*. Pada menu reservasi

(kasus menu 2), pemanggilan metode kontrol dibungkus secara rapi dalam struktur penanganan bertingkat. Setiap jenis objek exception ditangkap secara spesifik oleh blok catch-nya masing-masing, sehingga sistem mampu merespon dengan mencetak pesan galat yang personal, mendetail, dan ramah sesuai dengan jenis kekeliruan yang diperbuat oleh pengguna. Sesuai dengan spesifikasi instruksi penutupan aplikasi, blok finally diletakkan di akhir susunan try-catch utama menu. Blok ini memegang peranan absolut dalam menjamin bahwa kode pembersihan *resource* sistem (seperti konfirmasi penutupan sesi database) akan 100% pasti dieksekusi oleh mesin Java Virtual Machine (JVM), baik ketika program berjalan mulus tanpa hambatan maupun setelah program melewati serangkaian penanganan error yang bertubi-tubi.

IV. Kesimpulan

Berdasarkan hasil perancangan, implementasi, serta seluruh rangkaian uji coba simulasi yang telah dilakukan terhadap prototipe Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS", dapat disimpulkan bahwa penerapan mekanisme Error Handling menggunakan blok struktural try-catch-finally terbukti secara mutlak mampu meningkatkan robustness atau ketahanan aplikasi berbasis Command Line Interface (CLI). Sistem terbukti kebal dari potensi crash atau terminasi mendadak akibat kelalaian pengguna (human error), seperti kesalahan fatal saat memasukkan elemen alfabet pada kolom menu numerik ataupun kuantitas tiket. Melalui penanganan taktis berupa pembersihan input buffer memori lewat instruksi `input.nextLine()`; di dalam catch `InputMismatchException`, sistem sukses mengeliminasi risiko terjadinya fenomena infinite loop yang sering kali melumpuhkan fungsionalitas interaksi objek Scanner. Di sisi lain, pemisahan penanganan galat menjadi tiga objek custom exception memberikan granularitas dan kendali alur logika bisnis yang sangat ketat. Penggunaan `Unchecked Exception` berupa `DataPenumpangTidakValidException` menjadi instrumen filter yang ideal untuk menegakkan aturan mutlak pada format NIK penumpang secara instan sebelum data diproses lebih jauh oleh sistem. Sementara itu, penggunaan `Checked Exception` melalui `RuteTidakDitemukanException` dan `TiketHabisException` secara disiplin memaksa arsitektur program untuk selalu mengantisipasi ketidakpastian kondisi data di memori, baik ketika kode perjalanan tidak terdaftar maupun saat terjadi kondisi overcapacity kuota kursi kereta.

Melalui modifikasi khusus pada kelas `TiketHabisException`, objek eror kini tidak hanya berfungsi sebagai penanda kegagalan operasional, melainkan juga bertindak sebagai media pengantar data (data carrier) yang membawa informasi state aktual mengenai nama kereta dan sisa kuota kursi yang tersedia di memori. Karakteristik ini memfasilitasi sistem antarmuka untuk menyajikan pesan penolakan yang sangat informatif, transparan, dan ramah pengguna tanpa harus mengekspose variabel internal objek secara serampangan. Terakhir, implementasi blok finally di akhir siklus menu utama memberikan jaminan eksekusi absolut dari mesin Java Virtual Machine (JVM) saat pengguna memilih untuk keluar dari aplikasi. Blok ini berhasil memastikan bahwa seluruh proses krusial pasca transaksi seperti simulasi pemutusan koneksi basis data, pengosongan alokasi memori, serta pencetakan log penutupan sesi dapat terlaksana dengan aman dan konsisten, baik ketika aplikasi berjalan mulus tanpa hambatan maupun setelah sistem melewati rentetan penanganan interupsi eror yang bertubi-tubi. Secara keseluruhan, perancangan arsitektur ini berhasil melahirkan sebuah sistem kontrol reservasi lokal yang andal, aman, transparan, dan memenuhi seluruh spesifikasi standar rekayasa perangkat lunak.

V. Daftar Pustaka

Oracle. (n.d.). *Scanner (Java Platform SE 8)*. Oracle Documentation.

Oracle. (n.d.). *Assignment, Arithmetic, and Unary Operators*. The Java™ Tutorials.

Schildt, H. (2018). *Java: The Complete Reference*. McGraw-Hill Education.